$ desenvolvimento-mobile / aula-01

Do zero ao PWA até a loja

Criar o projeto, pacotes, manifesto, ícones, service worker, deploy e geração de APK/AAB.

Vue 3 · Vite · vite-plugin-pwa · Workbox · PWABuilder · TWA

Prof. Eduardo da Silva · IFC

ROTEIRO

O caminho completo

1. Criar
npm create vue
→
2. Pacotes
vite-plugin-pwa
→
3. Manifesto
+ ícones
→
4. Service Worker
cache offline

5. Testar
Lighthouse
→
6. Deploy HTTPS
domínio público
→
7. APK / AAB
TWA / PWABuilder
→
8. Loja
Google Play

Um único código-fonte web. A "versão nativa" é só um empacotamento do PWA já publicado.

01

Criar o projeto

Vue 3 + Vite, o ponto de partida do curso

Gerando o esqueleto

terminal copiar

```
$ npm create vue@latest registro-atividades-pwa
$ cd registro-atividades-pwa
$ npm install
$ npm run dev -- --host
```

Opções do assistente (comuns para os projetos desenvolvidos em aula):

- TypeScript, JSX, Vitest, E2E: **No**
- Router: **Yes** · Pinia: **Yes**
- ESLint: **Yes**

Por que `--host`?

Expõe o servidor na rede local. Abra `http://IP-do-PC:5173` no celular para testar o layout real.

Atenção

Service worker só funciona em **HTTPS** ou `localhost`. No IP da rede, o PWA não instala.

Nota

`registro-atividades-pwa` é o **nome do projeto**. Pode ser outro, desde que você use o mesmo nome nos comandos seguintes.

Estrutura de pastas

registro-atividades-pwa/ copiar

```
├── public/
│   ├── favicon.ico    # ícone da aba do navegador
│   └── icons/         # ícones do PWA (slide 11)
├── src/
│   ├── api/
│   ├── stores/
│   ├── views/
│   ├── components/
│   ├── router/
│   ├── assets/logo.svg  # logo do Vue (não usado)
│   ├── App.vue
│   └── main.js
├── index.html
├── vite.config.js   # config do PWA
└── package.json
```

`public/`

Arquivos servidos na raiz do site: ícones, `robots.txt` e, mais tarde, `.well-known/assetlinks.json`.

`vite.config.js`

Manifesto e service worker são **gerados no build** a partir daqui. Não escrevemos o SW na mão.

02

Pacotes

O que instalar e para quê

Dependências do PWA

PACOTE	TIPO	FUNÇÃO
`vite-plugin-pwa`	dev	Gera `manifest.webmanifest` e o service worker no build
`@bubblewrap/cli`	global	**Opcional.** Empacota o PWA em APK/AAB (TWA); só para quem quiser explorar (slide 22)

terminal copiar

```
$ npm i -D vite-plugin-pwa
```

O `workbox-build` (motor que gera o SW) vem como dependência do `vite-plugin-pwa`. O registro do SW usa `virtual:pwa-register`, fornecido pelo próprio plugin.

vite.config.js

vite.config.js — copiar

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { VitePWA } from 'vite-plugin-pwa'

export default defineConfig({
  plugins: [vue(), VitePWA({
    registerType: 'autoUpdate',
    manifest: {
      name: 'Registro de Atividades',
      short_name: 'Atividades',
      start_url: '/',
      display: 'standalone',
      theme_color: '#42b883',
      icons: [ /* próximos slides */ ],
    },
    workbox: { globPatterns: ['**/*.{js,css,html,svg,png}'] },
  })],
})
```

registerType

`autoUpdate` (o nosso): atualiza sozinho.
`prompt`: pergunta ao usuário.

globPatterns

O que entra no **precache**: o app inteiro fica disponível offline.

background_color

Cor da splash. Se omitir o campo, o plugin não inventa um valor.

Campos do manifesto

CAMPO	PARA QUÊ	EXEMPLO
`name` / `short_name`	Nome na instalação / sob o ícone (≤ 12 caracteres)	Registro de Atividades / Atividades
`start_url`	Página aberta ao tocar no ícone	`/`
`scope`	URLs consideradas "dentro" do app	`/`
`display`	Aparência: sem barra do navegador	`standalone`
`theme_color`	Cor da barra de status	`#42b883`
`background_color`	Cor da splash screen	`#ffffff`
`icons`	Ícones em vários tamanhos (obrigatório 192 e 512)	próximos slides
`orientation`	Trava a orientação (opcional)	`portrait`

O plugin gera `manifest.webmanifest` e injeta o `<link rel="manifest">` no HTML.

03

Ícones

Um SVG vira todos os tamanhos

Gerando os ícones

1. Acesse **pwabuilder.com/imageGenerator**
2. Envie uma imagem quadrada (ideal: 512×512 ou maior)
3. Escolha a plataforma (Android) e gere
4. Baixe o `.zip` e copie os ícones de **192×192** e **512×512** para `public/icons/`

public/icons/ copiar

```
public/
└── icons/
    ├── icon-192x192.png
    └── icon-512x512.png
```

ARQUIVO	USO
`icon-192x192.png`	Android / Chrome, tela inicial
`icon-512x512.png`	Splash, loja e ícone maskable

Ícone maskable e safe zone

vite.config.js (manifest.icons) copiar

```
icons: [
  { src: '/icons/icon-192x192.png',
    sizes: '192x192', type: 'image/png' },
  { src: '/icons/icon-512x512.png',
    sizes: '512x512', type: 'image/png' },
  {
    src: '/icons/icon-512x512.png',
    sizes: '512x512',
    type: 'image/png',
    purpose: 'maskable',
  },
]
```



O Android recorta em círculo, squircle, etc.
Mantenha o logo nos **80% centrais** (safe zone).

Teste em **maskable.app** antes de publicar.

index.html: o que falta no <head>

index.html

copiar

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta name="theme-color" content="#42b883" />
  <link rel="icon" href="/favicon.ico" sizes="any" />
  <link rel="apple-touch-icon" href="/icons/icon-192x192.png" />
  <title>Registro de Atividades</title>
</head>
```

theme-color

Repete a cor do manifesto para a barra do navegador antes do app instalar.

apple-touch-icon

O Safari/iOS **ignora** o ícone do manifesto, então precisa dessa tag.

Cache: escolha a estratégia certa

`precache`

App shell

JS, CSS, HTML e ícones. Baixados na instalação e servidos do cache. É o que faz o app abrir offline.

`NetworkFirst`

Dados da API

Tenta a rede, cai no cache se falhar. Bom para listas que mudam (`/tasks`).

`StaleWhileRevalidate`

Imagens, fontes

Responde do cache na hora e atualiza por baixo. Ótimo para recursos pouco críticos.

Outras: `CacheFirst` (assets imutáveis), `NetworkOnly` (login, pagamentos) e `CacheOnly`.

Runtime caching no Workbox

vite.config.js (VitePWA → workbox) copiar

```
workbox: {
  runtimeCaching: [
    {
      urlPattern: ({ url }) => url.pathname.startsWith('/tasks'),
      handler: 'NetworkFirst',
      options: {
        cacheName: 'api-tasks',
        networkTimeoutSeconds: 4,
      },
    },
    {
      urlPattern: ({ request }) => request.destination === 'image',
      handler: 'StaleWhileRevalidate',
      options: { cacheName: 'imagens' },
    },
  ],
},
```

`networkTimeoutSeconds: 4`: se a rede demorar mais de 4 s, usa o cache. É assim que o app "parece rápido" com sinal ruim.

Atualização do app

src/main.js copiar

```
import { registerSW } from 'virtual:pwa-register'

registerSW({
  immediate: true,
  onRegisteredSW(swUrl, registration) {
    if (registration) {
      setInterval(() => {
        registration.update()
      }, 60 * 1000) // checa a cada 60 s
    }
  },
})
```

Com `autoUpdate`

Deploy novo → o SW novo é baixado → assume o controle e a página recarrega, **sem ação do usuário**.

Alternativa

`registerType: 'prompt'` + componente `UpdatePrompt` (`useRegisterSW`) para perguntar antes. Indicado para formulários longos.

Testando de verdade

terminal copiar

```
$ npm run build
$ npm run preview
# abra http://localhost:4173
```

Por que build?

Em dev o SW é parcial. Teste sempre o **build de produção**.

1. DevTools → **Application** → **Manifest**
   sem avisos vermelhos, ícones carregando
2. **Service Workers**: status "activated"
   marque Offline e recarregue
3. **Lighthouse** → categoria PWA/Installable
   aprova os critérios de instalação
4. Instalar pelo ícone da barra de endereço
   abre em janela própria, sem abas

Publicar em HTTPS

GitHub Pages

Gratuito. Atenção ao `base` do Vite se o site ficar em `/repositorio/`.

Netlify / Vercel

Conectou o repositório, publica. Domínio `.app` grátis e HTTPS automático.

Servidor próprio

Nginx + Let's Encrypt. Fallback de rota para `index.html`.

Fabroku: plataforma que usamos nos projetos do IFC para publicar as aplicações.

Pegadinhas que quebram o PWA

- Rotas do Vue Router sem fallback → 404 ao recarregar
- Domínio de teste que muda a cada deploy → o APK aponta para um domínio fixo
- Servir `manifest.webmanifest` e `sw.js` com cache longo → atualização não chega

Para a loja você precisa de uma **URL estável**. Ela será o conteúdo do aplicativo.

07

APK, AAB e lojas

Empacotando o PWA para o Android

Que tipo de artefato?

caminho desta aula

TWA: Trusted Web Activity

- O Chrome abre seu site em tela cheia, sem barra
- É o **mesmo** PWA: deploy web atualiza o app
- APK minúsculo (~1–2 MB)
- Ferramentas: **PWABuilder** (recomendado) ou Bubblewrap (opcional)

alternativa

Capacitor (WebView)

- Embute o build `dist/` dentro do APK
- Plugins nativos: câmera, push, arquivos
- Exige Android Studio e atualiza só pela loja

ARTEFATO	PARA
`.apk`	Instalar direto no celular (testes, distribuição por link)
`.aab` (App Bundle)	**Obrigatório** para novos apps na Google Play

Caminho recomendado: PWABuilder

`recomendado` **TWA (Trusted Web Activity)**: um app Android mínimo que abre o *seu site* no Chrome em tela cheia, sem barra de endereço. Não há código web dentro do APK; ele só aponta para a URL publicada.

1. Acesse **pwabuilder.com** e informe a URL publicada
2. Revise o relatório (manifesto, SW, segurança)
3. **Package for stores → Android**
4. Preencha *Package ID*, nome, versão e **Signing key: New**
5. Baixe o `.zip`

O zip contém

- `app-release-signed.apk` para testar
- `app-release-bundle.aab` para a Play Store
- `signing.keystore` + `signing-key-info.txt`
- `assetlinks.json` pronto

Guarde o keystore!

Sem a chave de assinatura (ou sem a de upload cadastrada), você **não consegue publicar atualizações**. Faça backup fora do repositório.

Opcional: Bubblewrap CLI

opcional Não é necessário fazer na aula. Só para quem tiver interesse em explorar; o PWABuilder já resolve.

terminal copiar

```
$ npm i -g @bubblewrap/cli
$ mkdir android-twa && cd android-twa

# pergunta applicationId, nome, cores,
# ícone, versão e cria a keystore
$ bubblewrap init \
    --manifest=https://app.exemplo.com/manifest.webmanifest

# compila e assina
$ bubblewrap build
#  → app-release-signed.apk
#  → app-release-bundle.aab

# instala no celular (depuração USB)
$ adb install app-release-signed.apk
```

1ª execução

Oferece baixar o JDK e o Android SDK sozinho. Aceite.

applicationId

Identidade definitiva na loja. Ex.: `br.edu.ifc.atividades`. **Não muda depois.**

Nova versão

Aumente `appVersionCode` no `twa-manifest.json` e rode `bubblewrap update` + `build`.

Digital Asset Links: tirando a barra de URL

public/.well-known/assetlinks.json — copiar

```
[{
  "relation": ["delegate_permission/common.handle_all_urls"],
  "target": {
    "namespace": "android_app",
    "package_name": "br.edu.ifc.atividades",
    "sha256_cert_fingerprints": [
      "AA:BB:CC:...:99"
    ]
  }
}]
```

terminal — copiar

```
$ keytool -list -v -keystore android.keystore
# copie o "SHA256:" do certificado
```

sempre necessário

O que é

Prova de que **o site** autoriza **este app** a abrir em modo TWA. Sem ela, aparece a barra de URL. Vale para PWABuilder e Bubblewrap (não para Capacitor). O PWABuilder entrega o arquivo pronto: salve em `public/.well-known/`.

Erro nº 1

Com **Play App Signing**, a Google reassina o app. Use o SHA-256 do *Play Console → Integridade do app*, além do da sua chave.

Deve responder em `https://seudominio/.well-known/assetlinks.json` com `application/json`.

Alternativa: Capacitor

atenção Este slide **não é um passo a passo completo**: faltam configurações (ícones, splash, permissões, assinatura etc.). Requer mais estudo; veja **capacitorjs.com/docs**.

terminal copiar

```
$ npm i @capacitor/core @capacitor/android
$ npm i -D @capacitor/cli
$ npx cap init "Atividades" br.edu.ifc.atividades --web-dir=dist

$ npm run build          # gera dist/
$ npx cap add android
$ npx cap sync           # copia dist/ para o projeto Android
$ npx cap open android   # abre o Android Studio
# Build → Generate Signed Bundle / APK
```

Use quando

Precisa de recursos nativos que o navegador não dá (Bluetooth, contatos, push nativo) ou de funcionar 100% embutido.

Custo

Mais peso, Android Studio na máquina e **cada mudança passa por nova versão na loja**.

Google Play: o que preparar

Conta e app

- obrigatório Conta de desenvolvedor (taxa única)
- obrigatório Criar app e enviar o `.aab`
- recomendado Faixa de **teste interno** primeiro

Ficha da loja obrigatório

- Ícone 512×512
- Imagem de destaque 1024×500
- 2+ capturas de tela (celular)
- Descrição curta e completa

Conformidade obrigatório

- **Política de privacidade** em URL pública
- Declaração de segurança dos dados
- Classificação etária

Onde aprender: **Central de Ajuda do Play Console** (support.google.com/googleplay/android-developer) e **Políticas do Programa para Desenvolvedores** (play.google.com/about/developer-content-policy).

Contas pessoais novas

- Exigem **teste fechado** com testadores por ~14 dias antes de produção
- O *targetSdk* mínimo muda todo ano

Confira os valores atuais no Play Console.

E o iOS?

grátis

Adicionar à Tela de Início

No Safari: Compartilhar → **Adicionar à Tela de Início**. O PWA funciona com ícone e tela cheia, usando as tags `apple-touch-icon` do slide 13.

pago

App Store

Exige conta Apple Developer (anual), Mac com Xcode e um app com valor além de um "site embrulhado" (diretriz 4.2). PWABuilder gera o projeto Xcode, mas a aprovação não é garantida.

Na prática: Android via TWA na Play Store; iOS via instalação pelo Safari.

07 · MANUTENÇÃO

Atualizações depois de publicado

O QUE MUDOU	TWA	CAPACITOR
Código web (Vue, CSS, API)	**Só deploy web.** O app abre o site novo	Rebuild + `cap sync` + nova versão na loja
Nome, ícone, cor do app	Novo APK/AAB (`versionCode` +1)	Novo APK/AAB
Permissões, recursos nativos	Novo APK/AAB	Novo APK/AAB

Dica: guarde `appVersionCode` e `appVersionName` no repositório (sem o keystore!).

RESUMO

Checklist de ponta a ponta

- ☐ Projeto criado e rodando em `--host`
- ☐ `vite-plugin-pwa` instalado e configurado
- ☐ Manifesto completo (nome, cores, `display`)
- ☐ Ícones 192 / 512 / **maskable** / apple-touch
- ☐ SW com precache + runtime cache da API
- ☐ `registerSW` (atualização automática) no `main.js`
- ☐ Build testado offline + Lighthouse OK
- ☐ Deploy em HTTPS com URL estável
- ☐ `.aab` gerado (PWABuilder)
- ☐ `assetlinks.json` com o SHA-256 correto
- ☐ Keystore com backup seguro
- ☐ Ficha da loja + política de privacidade

Referências: web.dev/learn/pwa · vite-pwa-org.netlify.app · pwabuilder.com